

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación

Carrera de Ingeniería de Software

Asignatura: Aplicaciones Distribuidas (ISR-701)

Unidad 3 – Bases de Datos Distribuidas

Código: TA-IND-02

**Análisis de Consistencia y Replicación en Bases de Datos
Distribuidas: el caso del Sistema de Control de
Laboratorios Informáticos (SCLI)**

Estudiante: Freddy Farinango

Equipo PFC: ForaCode

Proyecto Fin de Curso: Sistema de Control de Laboratorios Informáticos (SCLI)

Docente: Prof. Guerrero Ulloa G. C.

Período académico: 2026–2027

Fecha de entrega: 14/07/2026

Análisis de Consistencia y Replicación en Bases de Datos Distribuidas: el caso del Sistema de Control de Laboratorios Informáticos (SCLI)

Freddy Farinango
Equipo PFC ForaCode
Carrera de Ingeniería de Software
Universidad Técnica Estatal de Quevedo
Aplicaciones Distribuidas (ISR-701)

Resumen—El Sistema de Control de Laboratorios Informáticos (SCLI) del equipo ForaCode es una aplicación Spring Boot que gestiona reservas, asignaciones y asistencia sobre una única instancia de PostgreSQL. Este informe examina, a partir de la revisión directa del código fuente y la configuración del proyecto, el modelo de consistencia realmente implementado. La evidencia indica que el SCLI ofrece únicamente consistencia transaccional local bajo el nivel de aislamiento predefinido de PostgreSQL (*read committed*, al no encontrarse configuración explícita), sin ninguna estrategia de replicación demostrable: opera con una sola fuente de datos, transacciones gestionadas por Spring y sin mecanismos de concurrencia optimista o pesimista más allá de restricciones `UNIQUE` puntuales. Se contrastan dos alternativas – replicación primario–réplica síncrona y asíncrona – frente al modelo actual, con criterios de consistencia observada, aislamiento, disponibilidad, latencia, tolerancia a particiones, riesgo de pérdida de datos, recuperación y complejidad operativa. El análisis, enmarcado en CAP y PACELC, concluye que el modelo actual es suficiente para el contexto académico presente del PFC, pero no constituye una base óptima para un despliegue con exigencias reales de disponibilidad, y propone una estrategia híbrida de replicación como mitigación.

Palabras clave—consistencia transaccional, replicación de bases de datos, CAP, PACELC, PostgreSQL, Spring Boot

I. INTRODUCCIÓN

El SCLI es la aplicación desarrollada por el equipo ForaCode para automatizar la reserva, asignación y monitoreo de laboratorios informáticos en la UTEQ, junto con el registro de asistencia de estudiantes y docentes. Al tratarse de un sistema donde múltiples usuarios compiten por el mismo recurso físico – un laboratorio en un horario determinado –, las garantías que ofrece la capa de datos frente a operaciones concurrentes son un aspecto central de su corrección, no un detalle de implementación secundario.

En sistemas distribuidos, ninguna base de datos replicada ofrece simultáneamente consistencia fuerte, disponibilidad to-

tal y tolerancia a particiones de red [4]. Cuando además se replica el estado entre nodos, aparece un segundo compromiso, formalizado por PACELC, entre la consistencia y la latencia incluso en ausencia de fallos [1]. Estas dos tensiones no son abstracciones teóricas ajenas al SCLI: determinan qué ocurriría si dos docentes intentan reservar el mismo laboratorio en el mismo instante bajo una arquitectura replicada, y qué sucede hoy con la disponibilidad del sistema si el único servidor de base de datos deja de responder.

El objetivo de este informe es identificar, con evidencia extraída directamente del código fuente y de los archivos de configuración del SCLI, el modelo de consistencia y la estrategia de replicación efectivamente implementados, evaluarlos frente al dominio del sistema y contrastarlos con al menos dos alternativas técnicas. La pregunta que guía el análisis es: *¿es el modelo de consistencia elegido el óptimo para el dominio del sistema?* El documento se organiza en un marco teórico sobre modelos de consistencia y replicación, un análisis del modelo real del SCLI, una evaluación de alternativas, una discusión crítica que responde a la pregunta orientadora, y las conclusiones correspondientes.

II. MARCO TEÓRICO

A. Modelos de consistencia

La consistencia en sistemas distribuidos no es una propiedad binaria, sino un espectro de garantías ordenadas por su fuerza semántica [6]. En el extremo más fuerte, la *linealizabilidad* exige que toda operación parezca ejecutarse de forma instantánea en un punto entre su invocación y su respuesta, de modo que cualquier lector observe siempre el último valor escrito por cualquier nodo del sistema. La *consistencia secuencial* relaja esta exigencia temporal pero preserva un orden global único de operaciones compartido por todos los procesos. La *consistencia causal* exige preservar únicamente el orden entre operaciones causalmente relacionadas, permitiendo que operaciones concurrentes se observen en órdenes distintos en distintas réplicas. La garantía de *lectura de los propios cambios* asegura que un proceso jamás deje de ver sus propias

escrituras previas, aun cuando no se garantice nada sobre las escrituras de terceros. Finalmente, la *consistencia eventual* solo garantiza que, en ausencia de nuevas escrituras, todas las réplicas convergerán eventualmente al mismo estado, sin acotar cuánto tiempo tomará esa convergencia.

Es importante no confundir este espectro, que describe garantías *entre réplicas*, con las propiedades ACID y los niveles de aislamiento de un motor transaccional que opera sobre una única instancia. La linealizabilidad es una garantía sobre el comportamiento observable de operaciones concurrentes, según la cual cada operación parece ejecutarse de manera instantánea en un punto comprendido entre su invocación y su respuesta; en sistemas replicados, preservarla exige coordinación entre nodos para mantener un orden compatible con el tiempo real. El aislamiento *read committed*, por ejemplo, impide que una transacción lea datos no confirmados por otra, pero no garantiza que distintas transacciones observen un único estado global inmutable ni elimina anomalías como lecturas no repetibles; *serializable* sí ofrece esa garantía dentro de un solo nodo, pero ninguno de los dos niveles equivale por sí mismo a linealizabilidad. Cuanto más fuerte es el modelo de consistencia distribuida, mayor es la coordinación necesaria entre nodos antes de confirmar una operación, lo que incrementa la latencia y reduce la disponibilidad bajo fallos; cuanto más débil es el modelo, menor es la coordinación requerida, a costa de exponer al cliente lecturas potencialmente obsoletas.

B. CAP y PACELC

El teorema CAP demuestra que, ante una partición de red, un sistema distribuido *replicado* no puede garantizar simultáneamente consistencia y disponibilidad; debe sacrificar una de las dos mientras la partición persiste [4]. Una lectura frecuente pero imprecisa reduce CAP a “elegir dos de tres propiedades en todo momento”; en realidad, la tolerancia a particiones no es opcional en un sistema verdaderamente distribuido con más de un nodo, por lo que la elección real ocurre únicamente entre C y A, y únicamente mientras dura la partición. Cuando solo existe un nodo de datos, no hay una segunda réplica con la cual particionarse, y CAP simplemente no se aplica como clasificación binaria; el problema relevante en ese caso es otro, y se retoma en la Sección III.

PACELC extiende esta idea al caso, mucho más frecuente en la operación cotidiana, en que no existe partición: incluso sin fallos de red, un sistema replicado debe elegir entre menor latencia y mayor consistencia, porque coordinar réplicas para garantizar linealizabilidad exige intercambios de mensajes adicionales [1]. Sistemas de referencia industrial ilustran ambos extremos: Dynamo prioriza la disponibilidad mediante replicación sin líder y consistencia eventual [3], mientras que Spanner ofrece consistencia externa apoyándose en sincronización temporal de alta precisión entre centros de datos [2].

C. Estrategias de replicación

La replicación líder–seguidor (primario–réplica) designa un único nodo que acepta escrituras y uno o más nodos que reciben los cambios de forma síncrona o asíncrona. En el modo síncrono, el líder espera la confirmación de al menos una réplica antes de reconocer la escritura al cliente, lo que evita la pérdida de datos ante la caída del líder a costa de mayor latencia y del riesgo de bloquear escrituras si la réplica no responde. En el modo asíncrono, el líder confirma la escritura sin esperar a la réplica, reduciendo la latencia percibida pero exponiendo una ventana de posible pérdida de transacciones recientes y de lecturas obsoletas (*staleness*, entendida aquí como el retraso entre el momento en que un dato se escribe en el líder y el momento en que se refleja en una réplica) si el líder falla antes de propagar el cambio. Trabajos recientes muestran que sostener consistencia fuerte entre réplicas geográficamente distribuidas tiene un costo de coordinación medible y no trivial, que debe compararse explícitamente contra los requisitos reales de la aplicación antes de adoptarse [10].

La replicación multi-líder permite que varios nodos acepten escrituras simultáneamente, lo que exige resolución de conflictos cuando dos líderes modifican el mismo dato de forma concurrente. La replicación sin líder, empleada por sistemas como Dynamo, distribuye lecturas y escrituras entre un conjunto de réplicas y utiliza *quorums* de lectura y escritura para acotar las garantías de consistencia observadas [3]. Las arquitecturas líder–seguidor, multi-líder y sin líder presentan diferentes compromisos de coordinación, disponibilidad y resolución de conflictos [5]. La elección entre estas estrategias continúa siendo un área activa de investigación aplicada, particularmente cuando las transacciones deben mantenerse eficientes sin renunciar a garantías de aislamiento en escenarios multi-región [8].

D. Consistencia de datos en arquitecturas orientadas a servicios

La adopción de arquitecturas donde cada servicio administra su propia base de datos traslada buena parte de la responsabilidad de mantener la consistencia desde el motor de base de datos hacia la capa de aplicación. Un relevamiento reciente de prácticas industriales documenta que los equipos de desarrollo enfrentan de forma recurrente problemas de integridad referencial entre servicios, ausencia de soporte transaccional nativo entre bases de datos independientes y falta de herramientas maduras para depurar fallos de consistencia entre servicios [7]. Este hallazgo es relevante para el SCLI porque, aunque el sistema no adopta actualmente una descomposición en microservicios, cualquier evolución futura hacia esa arquitectura heredaría estas mismas tensiones. Un modelo más amplio de los sistemas de datos distribuidos permite además situar al SCLI dentro de un panorama más general de estrategias de sincronización, sean estas basadas en transacciones distribuidas, en propagación de eventos o en replicación directa del motor de base de datos [9].

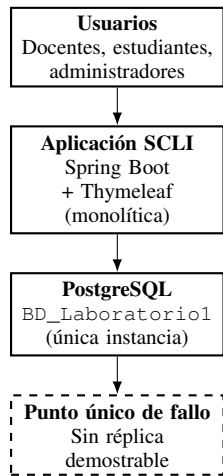


Fig. 1. Topología actual del SCLI: aplicación monolítica conectada a una única instancia PostgreSQL, sin réplicas demostrables.

III. ANÁLISIS DEL MODELO DE CONSISTENCIA Y REPLICACIÓN DEL SCLI

A. Arquitectura real identificada

La revisión del proyecto entregado (pom.xml, directorio src/main/java/com/uteq/SCLI) confirma que el SCLI es una **aplicación monolítica única** construida con Spring Boot 3.5.4, con un solo módulo Maven (groupId com.uteq, artifactId SCLI) y sin submódulos ni descriptores de microservicios. No se encontró ningún Dockerfile ni docker-compose.yml en el proyecto entregado, por lo que no puede afirmarse la existencia de una topología de despliegue de múltiples nodos o contenedores; tampoco se dispone de scripts DDL, migraciones versionadas ni archivos de configuración del servidor PostgreSQL (postgresql.conf, pg_hba.conf), por lo que cualquier afirmación sobre el motor de base de datos se limita a lo observable desde la aplicación.

El archivo application.properties define la propiedad spring.datasource.url con el valor jdbc:postgresql://localhost:5432/BD_Laboratorio1, con credenciales tomadas de variables de entorno (DB_USER, DB_PASS, valores omitidos por seguridad). No existen en el proyecto clases de configuración de múltiples DataSource, ni perfiles que apunten a hosts distintos: los perfiles application-dev.properties y application-prod.properties únicamente ajustan spring.jpa.hibernate.ddl-auto y el caché de Thymeleaf, sin introducir un segundo nodo de base de datos. En consecuencia, el SCLI mantiene una **única fuente de datos**: una sola instancia de PostgreSQL, sin réplicas de lectura ni nodos de respaldo demostrables en los archivos revisados.

Como se observa en la Fig. 1, el SCLI depende de una única instancia PostgreSQL y no presenta una réplica demostrable en los archivos revisados.

B. Transacciones y mecanismos de concurrencia

Se identificaron 20 archivos con anotaciones @Transactional, distribuidas entre repositorios (por ejemplo AdminUsuarioRepository, PersonaRepository), servicios (AuthService, SolicitudReservaService, LaboratorioService, entre otros) y el controlador HorarioApiController. Estas anotaciones delimitan transacciones ACID locales a la instancia única de PostgreSQL, gestionadas por el motor transaccional de Spring; no implican, por sí mismas, ninguna forma de replicación ni de consistencia distribuida. No se encontró ninguna configuración explícita de nivel de aislamiento (no hay ocurrencias de isolation en las anotaciones revisadas), por lo que las transacciones utilizan previsiblemente el nivel predeterminado de PostgreSQL, *read committed*. Este nivel impide que una transacción lea datos no confirmados por otra, pero no equivale a linealizabilidad ni a consistencia fuerte distribuida, no garantiza que toda transacción observe un único estado inmutable a lo largo de su ejecución, y no elimina anomalías de concurrencia como las lecturas no repetibles.

Respecto a mecanismos de control de concurrencia adicionales, no se halló ninguna anotación @Version (bloqueo optimista de JPA), ninguna consulta SELECT ... FOR UPDATE, ninguna palabra clave synchronized ni uso de LockModeType en el código fuente revisado. Las únicas restricciones de integridad relevantes son UNIQUE a nivel de columna o de tabla: en Materia (cod_materia), Rol (nombre_rol), Usuario (nombre_usuario, y una relación OneToOne única con Persona), Persona (columna única declarada sin nombre explícito en la anotación) y Equipo (codigo_equipo). En las entidades JPA y archivos revisados no se encontró evidencia de una restricción UNIQUE compuesta sobre laboratorio, fecha y franja horaria. Sin embargo, al no disponer del DDL completo, de migraciones versionadas ni de la configuración real del servidor PostgreSQL, no puede descartarse que exista una restricción, disparador (trigger) o función implementada directamente en la base de datos y ausente en el código de aplicación. Las entidades de dominio más sensibles a este riesgo – AsignacionLaboratorio, SolicitudAsignacion y ReservaEspecial – no declaran, en el código Java revisado, ninguna restricción UNIQUE compuesta ni mecanismo de bloqueo; con la evidencia disponible, la prevención de condiciones de carrera en la asignación de laboratorios parece depender de la lógica de validación en la capa de servicio, sin que pueda confirmarse ni descartarse una salvaguarda adicional a nivel de motor.

Es relevante notar también el uso de set_config con ámbito de transacción (PgApplicationContextFilter, DbRoleResetFilter) para propagar el identificador del docente autenticado como variable de sesión de PostgreSQL, empleada presumiblemente por políticas de seguridad a nivel de fila. Este mecanismo es inherentemente local a la conexión

y a la transacción activa: en una arquitectura con réplicas, este contexto de sesión tendría que reestablecerse en cualquier nodo que atienda una consulta bajo esas políticas, lo cual es una consideración adicional para cualquier estrategia de replicación futura.

C. Criticidad de los datos por módulo

No todas las operaciones del SCLI exigen el mismo nivel de exigencia sobre la actualidad del dato leído. Las reservas, asignaciones de laboratorio, control de horarios, registro de asistencia, gestión de usuarios y de roles/permisos requieren controles de concurrencia que eviten dobles asignaciones o actualizaciones incompatibles. La existencia de una única fuente elimina la divergencia entre réplicas, pero el aislamiento *read committed* no evita por sí solo todas las condiciones de carrera: cada sentencia observa una instantánea de los datos confirmados al inicio de su ejecución, sin garantizar resultados serializables ante operaciones concurrentes sobre el mismo recurso. En contraste, los reportes agregados, los paneles informativos y las consultas analíticas no críticas (evidenciados en el paquete `controller.ReporteControllerFreddy` y las vistas de `dashboard/admin/reportes`) podrían tolerar datos ligeramente atrasados sin afectar la corrección funcional del sistema, siempre que el retraso se mantenga acotado y se comunique al usuario.

D. Lectura CAP/PACELC del modelo actual

Una arquitectura de nodo único no implementa, en sentido estricto, tolerancia a particiones: no existe un segundo nodo con el cual particionarse. Por ello no es correcto calificar al SCLI como “CP” o “AP”; ambas etiquetas presuponen una topología replicada, y CAP se vuelve decisivo únicamente cuando se introducen réplicas y aparece la posibilidad de una partición entre nodos de datos. Lo que sí puede afirmarse es que la caída de la única instancia de PostgreSQL produce la pérdida total de disponibilidad del sistema, sin que exista ningún nodo de respaldo que asuma el servicio. Esto constituye un **punto único de fallo** (*single point of failure*) tanto a nivel de datos como, potencialmente, a nivel de aplicación si esta se despliega en una sola instancia. Bajo PACELC, al no existir réplicas, el sistema no enfrenta el compromiso entre latencia y consistencia que sí aparecería al introducir replicación: la baja complejidad operativa actual y la ausencia de coordinación entre nodos tienen como contrapartida que cualquier falla de la única instancia detiene por completo el servicio.

E. Limitaciones de la evidencia disponible

El ZIP entregado no incluye scripts de creación del esquema (DDL) ni migraciones versionadas; la estructura de tablas se infiere únicamente de las anotaciones JPA de las entidades. Tampoco se incluyen archivos de configuración de PostgreSQL ni evidencia de un entorno de despliegue (contenedores, orquestación) más allá del código de la aplicación. El análisis presentado se limita, por tanto, a lo demostrable a partir del código fuente, las propiedades de Spring y los

archivos de configuración incluidos en el proyecto; donde la evidencia es incompleta, este informe lo indica explícitamente en lugar de asumir una ausencia absoluta.

IV. EVALUACIÓN DE ALTERNATIVAS

A. Modelo actual: instancia única

Como se describió en la Sección III, todas las lecturas del SCLI se dirigen a una única fuente de datos, por lo que no existe divergencia entre réplicas. El aislamiento *read committed* evita la lectura de datos no confirmados, pero no garantiza serialización de todas las operaciones concurrentes ni equivale a linealizabilidad. Esta última constituye una garantía distinta sobre el orden observable de las operaciones y, en una arquitectura replicada, requiere coordinación entre nodos. El costo del modelo actual es la ausencia total de tolerancia a fallos del nodo de datos.

B. Alternativa 1: replicación primario–réplica síncrona

En este modelo, el nodo primario espera la confirmación de al menos una réplica síncrona antes de reconocer la escritura. Para el SCLI, esto preservaría la consistencia transaccional ya existente en las operaciones críticas (reservas, asignaciones, asistencia, usuarios y permisos) mientras ofrece un nodo adicional con una copia de los datos ante la caída del primario. La incorporación de una réplica síncrona puede **reducir**, mas no eliminar por sí sola, el punto único de fallo del almacenamiento: para que efectivamente mejore la disponibilidad se requiere además detección de fallos, promoción controlada de la réplica a primario (que no ocurre automáticamente salvo que exista una herramienta de orquestación configurada para ello), redirección de conexiones de la aplicación hacia el nuevo primario, monitoreo continuo y procedimientos de *failover* probados periódicamente; de lo contrario pueden persistir puntos únicos de fallo adicionales en la aplicación, la red o el balanceador. El costo es una mayor latencia por confirmación de escritura y la posibilidad de que las escrituras se detengan o bloqueen si la réplica síncrona requerida no responde. Ante una partición de red entre el primario y su réplica síncrona, el sistema debería decidir explícitamente entre rechazar temporalmente las escrituras (preservando consistencia) o degradar a un modo de menor garantía (preservando disponibilidad); para operaciones de reserva y asignación, rechazar temporalmente la operación es preferible a permitir una doble asignación que luego deba revertirse manualmente. Como señalan estudios recientes sobre el costo de la consistencia fuerte en réplicas geo-distribuidas, esta coordinación adicional no es gratuita y debe dimensionarse contra el volumen real de escrituras concurrentes del sistema [10].

C. Alternativa 2: replicación primario–réplica asíncrona

Aquí el primario confirma la escritura sin esperar a la réplica, reduciendo la latencia de escritura percibida por el usuario. El riesgo es la pérdida de transacciones recientes si el primario falla antes de propagar el cambio, y que las lecturas dirigidas a la réplica reflejen un estado ligeramente atrasado. Esta alternativa resulta útil para descargar del nodo primario

las consultas de reportes, paneles informativos y estadísticas de uso – operaciones que, según el análisis de criticidad de la Sección III, toleran cierto grado de *staleness* – sin comprometer la consistencia transaccional de las operaciones críticas, que continuarían atendiéndose exclusivamente en el primario. Ninguna de las dos alternativas, por sí sola, garantiza alta disponibilidad: ambas requieren monitoreo continuo, procedimientos de recuperación probados y, en el caso síncrono, un mecanismo de *failover* verificado periódicamente.

D. Tabla comparativa

La Tabla I presenta los principales compromisos técnicos entre el modelo actual y las dos alternativas evaluadas. Ninguna celda debe interpretarse como una garantía absoluta: la disponibilidad de la replicación síncrona, por ejemplo, depende de que el mecanismo de *failover* esté correctamente configurado y probado, y no es automática por el solo hecho de introducir una réplica.

E. Recomendación argumentada

La arquitectura recomendada se representa en la Fig. 2. Una estrategia híbrida resulta más adecuada que adoptar un único esquema de replicación para todas las operaciones: réplica síncrona para las operaciones críticas identificadas en la Sección III (reservas, asignaciones, asistencia, usuarios, permisos) y réplica asíncrona – o consultas dirigidas a la misma réplica síncrona en modo de solo lectura – para reportes e indicadores no críticos. Esta propuesta no equivale por sí sola a alta disponibilidad: requiere infraestructura adicional para alojar al menos un nodo de réplica, administración y monitoreo continuo del estado de replicación, un procedimiento de *failover* documentado y probado periódicamente (no automático por defecto), restricciones de integridad adicionales a nivel de motor, y pruebas de recuperación ante fallos que hoy no existen en el proyecto revisado.

Tabla I
COMPARACIÓN ENTRE EL MODELO ACTUAL DEL SCLI Y DOS ALTERNATIVAS DE REPLICACIÓN.

Criterio	Modelo actual (instancia única)	Réplica síncrona	Réplica asíncrona
Consistencia observada	Transaccional local bajo <i>read committed</i> presumible; sin divergencia entre réplicas porque no existen réplicas	Transaccional local, extendida a la réplica confirmada antes del commit	Transaccional local en el primario; réplica con posible retraso
Aislamiento y concurrencia	<i>Read committed</i> presumible; sin bloqueo optimista ni pesimista explícito	Igual que el actual en el primario; agrega coordinación de commit	Igual que el actual en el primario; réplica de solo lectura
Disponibilidad	Nula ante caída del único nodo	Mejorada solo si el <i>failover</i> está configurado, monitoreado y probado	Mejorada para lecturas; la escritura sigue dependiendo del primario
Latencia de escritura	Baja (sin coordinación)	Mayor (espera confirmación de la réplica)	Baja (no espera a la réplica)
Tolerancia a particiones	No aplica (nodo único)	Requiere decidir entre rechazar escrituras o degradar el servicio	Favorece disponibilidad; la réplica puede quedar desactualizada
Riesgo de pérdida de datos	Bajo dentro del nodo único	Bajo (la réplica confirma antes del commit)	Existe una ventana entre el commit y la propagación
Recuperación ante fallos	Sin nodo de respaldo; requiere restaurar desde copia de seguridad	Failover hacia la réplica, si fue probado y monitoreado previamente	Failover posible, con riesgo de perder transacciones recientes
Complejidad operativa	Baja	Alta: monitoreo, promoción de réplica, pruebas periódicas de recuperación	Media: monitoreo del retraso de replicación
Adecuación al SCLI	Suficiente para el contexto académico actual del PFC	Adecuada para reservas, asignaciones y asistencia	Adecuada para reportes e indicadores no críticos

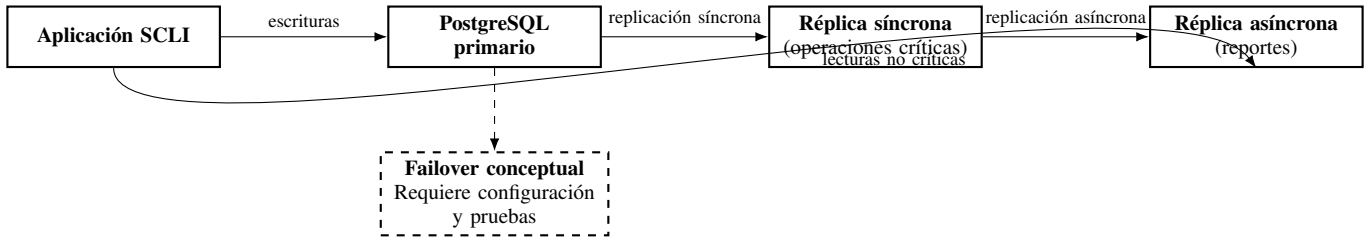


Fig. 2. Topología propuesta para el SCLI: réplica síncrona para operaciones críticas y réplica asíncrona para consultas no críticas.

V. DISCUSIÓN CRÍTICA

¿Es el modelo de consistencia elegido el óptimo para el dominio del sistema? La respuesta, a partir de la evidencia recogida, es **matizadamente negativa**: el modelo actual es razonable como solución transitoria para un proyecto académico en desarrollo, pero no constituye una base adecuada para un despliegue en producción que deba garantizar continuidad de servicio.

A favor del modelo actual puede reconocerse que ofrece consistencia transaccional local sin ningún costo de coordinación entre nodos: todas las consultas acceden a la misma instancia, por lo que no existe divergencia entre réplicas; no obstante, el comportamiento concurrente continúa condicionado por el aislamiento *read committed* y por los controles de integridad disponibles, y no por una propiedad de consistencia distribuida que el sistema no implementa. Esto simplifica considerablemente el desarrollo y es adecuado mientras el vol-

umen de usuarios concurrentes y la exigencia de disponibilidad continua se mantengan bajos, como corresponde a un sistema en fase de proyecto de curso. Sin embargo, sus limitaciones son sustanciales: el punto único de fallo identificado en la Sección III implica que cualquier interrupción del único servidor de PostgreSQL – por mantenimiento, saturación de recursos o fallo de hardware – deja al sistema completamente inoperativo, sin capacidad de degradar el servicio de forma controlada. Adicionalmente, no se encontró evidencia de restricciones *UNIQUE* compuestas ni de bloqueos explícitos en las entidades de reserva y asignación en el código Java revisado – sin que ello permita descartar de forma absoluta una salvaguarda a nivel de motor no visible en los archivos entregados –, lo cual es una condición frágil frente a picos de concurrencia reales, como el inicio de un período de matrículas.

Leído bajo CAP, el sistema no ejerce hoy ninguna decisión explícita entre consistencia y disponibilidad porque no hay réplicas con las cuales particionarse; esa decisión queda

pospuesta, no resuelta, y se volverá decisiva en cuanto se introduzca una segunda instancia de datos. Leído bajo PACELC, el sistema tampoco enfrenta hoy el compromiso entre latencia y consistencia propio de las arquitecturas replicadas, pero a cambio asume el riesgo máximo de disponibilidad. La mitigación concreta que se propone es la incorporación de al menos una réplica síncrona para las operaciones críticas, acompañada de restricciones `UNIQUE` compuestas (laboratorio, fecha, franja horaria) que blinden a nivel de motor de base de datos lo que hoy parece depender únicamente de la capa de aplicación. Para el contexto actual del PFC – entorno académico, volumen de usuarios acotado, sin obligación contractual de disponibilidad continua –, el modelo de instancia única es *suficiente*; para un eventual despliegue institucional real, con exigencias de continuidad de servicio durante períodos académicos críticos, no es óptimo.

VI. CONCLUSIONES

La revisión del proyecto permitió verificar que el SCLI implementa consistencia transaccional local sobre una única instancia de PostgreSQL, con aislamiento presumiblemente *read committed* al no encontrarse configuración explícita, y sin ninguna estrategia de replicación demostrable en `application.properties` ni en la ausencia de múltiples `DataSource`, `Dockerfile` o `docker-compose.yml` en el proyecto entregado. El aislamiento *read committed* evita la lectura de datos no confirmados, pero no equivale a linealizabilidad: esta última es una garantía distinta sobre el orden observable de las operaciones concurrentes que, en una arquitectura replicada, requeriría coordinación explícita entre nodos.

Frente a la pregunta orientadora, el modelo actual no es el óptimo para un dominio que involucra reservas concurrentes de recursos físicos limitados y que aspira a una eventual operación institucional continua: la ausencia de un nodo de respaldo constituye un punto único de fallo, y la ausencia de evidencia de restricciones a nivel de motor para combinaciones laboratorio–horario deja la integridad de las reservas dependiendo, hasta donde permite verificar la evidencia disponible, de la lógica de aplicación. La alternativa recomendada es una estrategia híbrida de replicación primario–réplica, síncrona para operaciones críticas y asíncrona (o de solo lectura sobre la misma réplica) para reportes, que preserva la consistencia donde es indispensable y mejora la disponibilidad general del sistema a un costo de complejidad operativa acotado, siempre que se acompañe de pruebas de *failover* y recuperación periódicas.

Este estudio se limita a lo verificable en el código fuente y la configuración entregados; no se dispuso de los scripts de esquema de base de datos ni de la configuración de despliegue real del PFC, por lo que cualquier decisión de arquitectura de replicación requeriría validarse adicionalmente contra el volumen real de usuarios concurrentes y los requisitos institucionales de continuidad de servicio del equipo ForaCode.

DECLARACIÓN DE USO DE INTELIGENCIA ARTIFICIAL GENERATIVA

Se utilizó Claude, de Anthropic, como herramienta de apoyo para organizar la estructura del documento, aclarar conceptos técnicos y revisar la redacción académica. El análisis del código fuente, la interpretación del modelo de consistencia, la validación de los hallazgos, la selección y comprobación de las referencias, la evaluación de alternativas y las conclusiones fueron revisadas y validadas por el autor. La herramienta no fue utilizada como fuente bibliográfica.

REFERENCIAS

- [1] D. J. Abadi, “Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story,” *Computer*, vol. 45, no. 2, pp. 37–42, 2012. doi: 10.1109/MC.2012.33
- [2] J. C. Corbett *et al.*, “Spanner: Google’s Globally Distributed Database,” *ACM Transactions on Computer Systems*, vol. 31, no. 3, pp. 1–22, 2013, art. no. 8. doi: 10.1145/2491245
- [3] G. DeCandia *et al.*, “Dynamo: Amazon’s Highly Available Key-value Store,” in *Proc. 21st ACM SIGOPS Symposium on Operating Systems Principles (SOSP ’07)*, 2007, pp. 205–220. doi: 10.1145/1294261.1294281
- [4] S. Gilbert and N. Lynch, “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services,” *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, 2002. doi: 10.1145/564585.564601
- [5] M. van Steen and A. S. Tanenbaum, *Distributed Systems*, 3rd ed. distributed-systems.net, 2017. ISBN 978-1543057386.
- [6] P. Viotti and M. Vukolić, “Consistency in Non-Transactional Distributed Storage Systems,” *ACM Computing Surveys*, vol. 49, no. 1, pp. 1–34, 2016, art. no. 19. doi: 10.1145/2926965
- [7] R. Laigner, Y. Zhou, M. A. Vaz Salles, Y. Liu, and M. Kalinowski, “Data Management in Microservices: State of the Practice, Challenges, and Research Directions,” *Proceedings of the VLDB Endowment*, vol. 14, no. 13, pp. 3348–3361, 2021. doi: 10.14778/3484224.3484232
- [8] C. D. T. Nguyen, J. K. Miller, and D. J. Abadi, “Detock: High Performance Multi-Region Transactions at Scale,” *Proceedings of the ACM on Management of Data*, vol. 1, no. 2, art. no. 148, 2023. doi: 10.1145/3589293
- [9] A. Margara, G. Cugola, N. Felicioni, and S. Cilloni, “A Model and Survey of Distributed Data-Intensive Systems,” *ACM Computing Surveys*, vol. 56, no. 1, 2023. doi: 10.1145/3604801
- [10] Y. Du, Z. Xu, K. Zhang, J. Liu, C. Stewart, and J. Huang, “Cost-Effective Strong Consistency on Scalable Geo-Diverse Data Replicas,” *IEEE Transactions on Cloud Computing*, vol. 11, no. 2, pp. 1764–1776, 2023. doi: 10.1109/TCC.2022.3161297